

INSTITUTO TECNOLÓGICO DE VERACRUZ

Nombre de la Asignatura:

- Administración en Base de Datos

Grupo: **AD26 6J3A**

Nombres del alumno:

- Raúl Alberto Guerrero Aguilar
- Num. Control: 23600266

Nombre de la Actividad:

- Reporte de Práctica v1

Carrera: Ingeniería en Sistemas Computacionales

Fecha de Entrega: 24/Septiembre/2026

1. Introducción y Preparación del Entorno

La práctica se desarrolló sobre una máquina virtual creada con Oracle VirtualBox, en la cual se instaló el sistema operativo Linux Mint. El uso de una máquina virtual permite aislar el entorno experimental del sistema anfitrión, de modo que cualquier modificación en paquetes, servicios o configuraciones de red pueda revertirse o descartarse sin afectar la instalación principal del equipo.

El objetivo consistió en instalar el motor de contenedores Podman y desplegar exitosamente dos contenedores en sus versiones gratuitas (free/community): MongoDB, como sistema gestor NoSQL orientado a documentos, y MySQL, como sistema gestor relacional (SQL), resolviendo los conflictos de puertos que se presentaran durante el proceso.

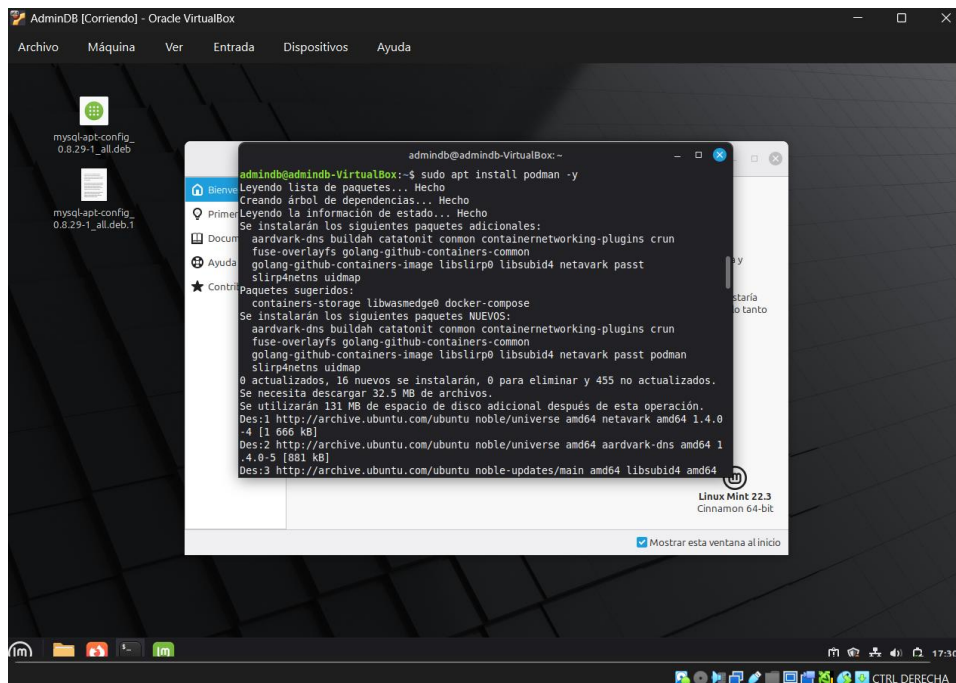
Se optó por Podman como motor de contenedores debido a su arquitectura **daemonless**, es decir, sin un demonio en segundo plano. A diferencia de Docker, cuyo funcionamiento depende de un servicio persistente (`dockerd`) que habitualmente se ejecuta con privilegios de superusuario y actúa como intermediario de todas las operaciones, Podman ejecuta cada contenedor como un proceso hijo del usuario que invoca el comando. Esta decisión de diseño elimina un punto único de falla, reduce la superficie de ataque asociada a un proceso privilegiado permanente y facilita la ejecución de contenedores sin privilegios de administrador (modo **rootless**). Adicionalmente, Podman conserva una interfaz de línea de comandos compatible con la de Docker y trabaja con imágenes bajo el estándar OCI, por lo que las imágenes y los conocimientos existentes son directamente reutilizables.

La instalación se realizó desde los repositorios oficiales de la distribución. En primer lugar se actualizó el índice local de paquetes, con el fin de que el gestor APT consulte las versiones y dependencias más recientes disponibles:

```
sudo apt update
```

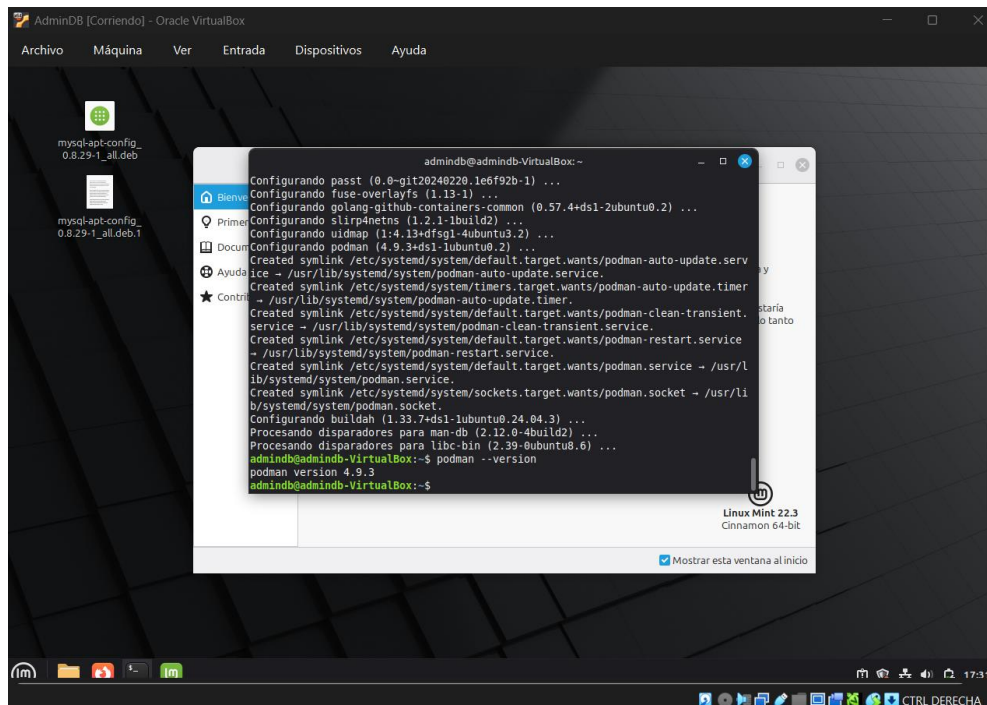
Posteriormente se instaló el paquete de Podman. La opción `-y` responde afirmativamente, de forma automática, a las solicitudes de confirmación del gestor de paquetes, lo que evita interrumpir el proceso:

```
sudo apt install podman -y
```



Una vez concluida la instalación, se verificó que el cliente respondiera correctamente y se consultó la versión instalada mediante el siguiente comando:

```
podman versión
```



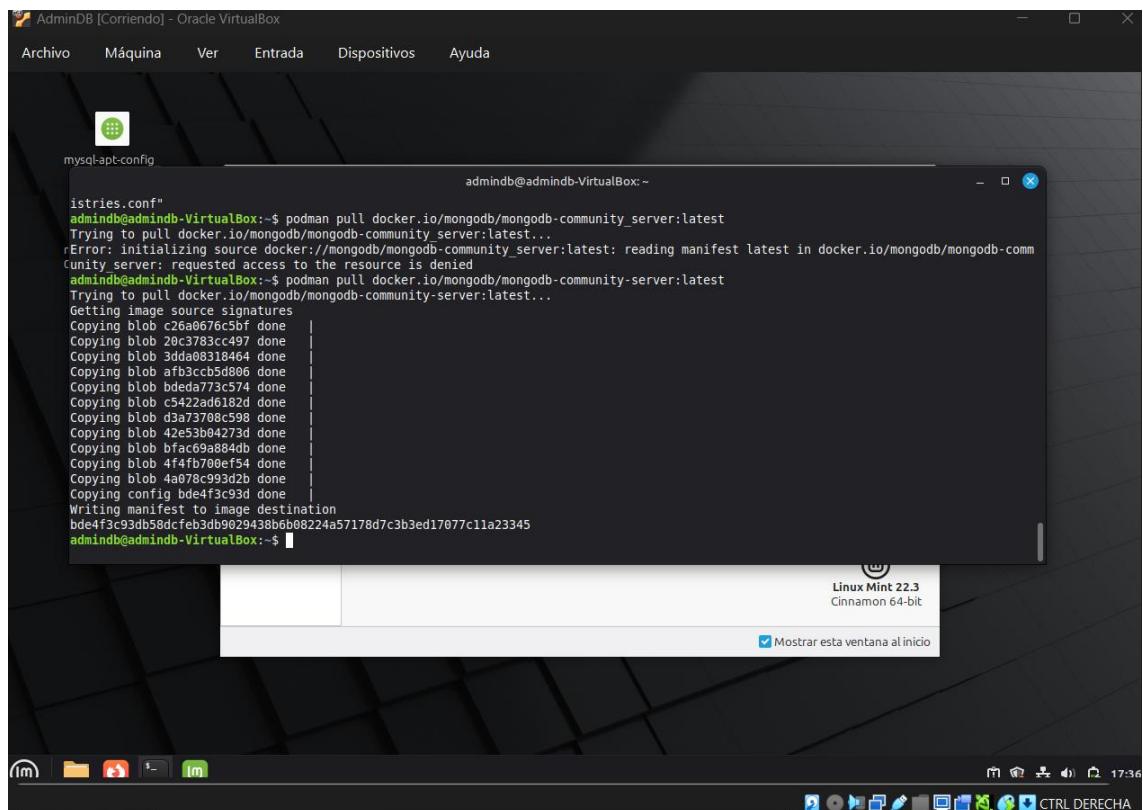
2. Despliegue de Base de Datos Orientada a Documentos (MongoDB)

El primer servicio desplegado fue MongoDB, para lo cual se descargó la imagen comunitaria desde Docker Hub. En el nombre de la imagen se especificó de forma explícita el registro (`docker.io`), dado que Podman no asume un único registro por defecto; indicarlo evita ambigüedades y la solicitud interactiva de selección de registro.

En el primer intento de descarga se cometieron dos errores de escritura: la etiqueta se escribió como `lastest` en lugar de `latest`, y el nombre del repositorio incluyó un guion bajo en lugar del guion medio correspondiente. Como el registro identifica las imágenes por coincidencia exacta de repositorio y etiqueta, el comando fue rechazado al no localizar el recurso solicitado. Una vez corregidos ambos errores, la descarga se completó exitosamente con el comando definitivo:

```
podman pull docker.io/mongodb/mongodb-community-server:latest
```

Una imagen OCI se compone de capas de solo lectura. Podman descarga cada una de ellas por separado, verifica su integridad mediante su resumen criptográfico (*digest*) y las almacena localmente, de manera que las capas compartidas entre distintas imágenes se reutilizan y no se descargan nuevamente.



Con la imagen disponible localmente, se levantó el contenedor en segundo plano mediante el siguiente comando:

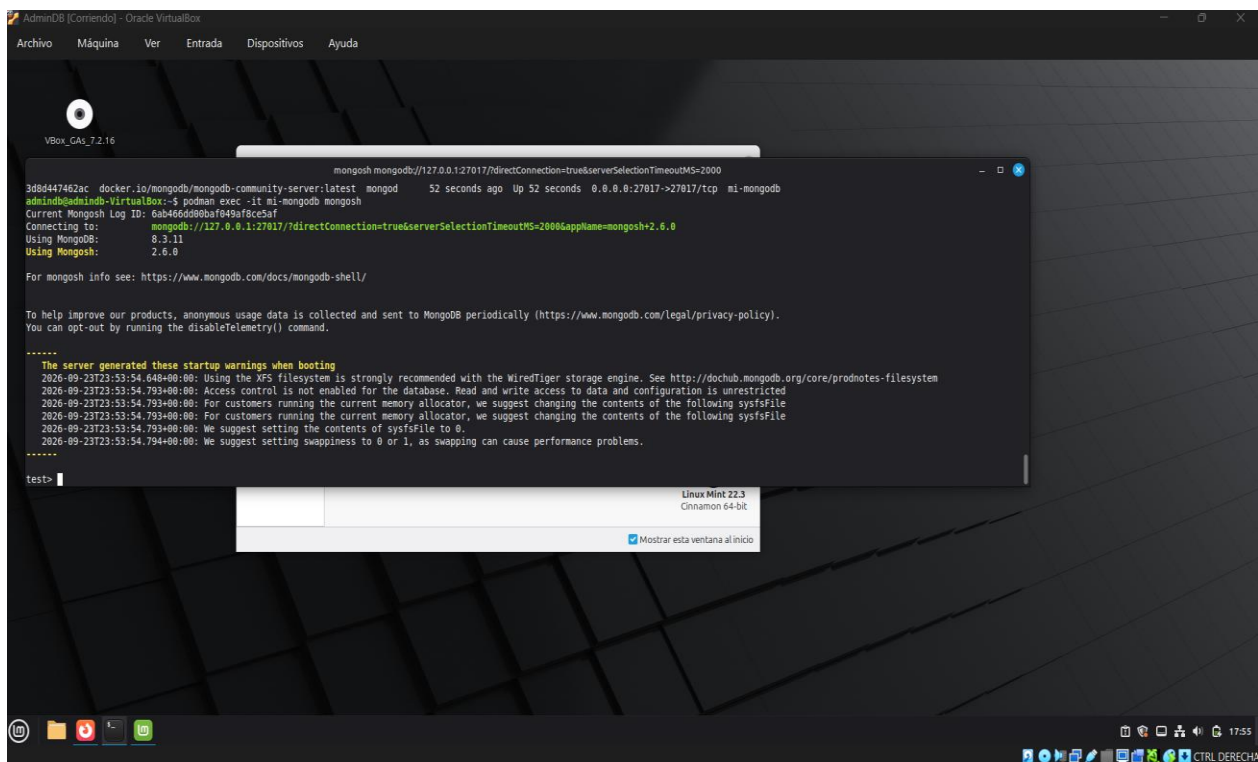
```
podman run --name mi-mongodb -d -p 27017:27017  
docker.io/mongodb/mongodb-community-server:latest
```

La opción `--name` asigna al contenedor un identificador legible (`mi-mongodb`), útil para referenciarlo en operaciones posteriores; la opción `-d` (*detached*) lo ejecuta en segundo plano, liberando la terminal; y la opción `-p 27017:27017` establece el mapeo de puertos con el formato puerto del anfitrión : puerto del contenedor. El puerto 27017 es el predeterminado de MongoDB, y su publicación en la interfaz del anfitrión es necesaria porque, de otro modo, el servicio solo sería accesible desde el espacio de red aislado del propio contenedor.

Para comprobar el funcionamiento del servicio, se accedió a la consola interactiva de la base de datos ejecutando un proceso adicional dentro del contenedor en ejecución:

```
podman exec -it mi-mongodb mongosh
```

El subcomando `exec` ejecuta un comando dentro de un contenedor activo; la opción `-i` mantiene abierta la entrada estándar y la opción `-t` asigna una pseudoterminal, lo que hace posible la interacción con el programa. Por su parte, `mongosh` es el intérprete de comandos oficial de MongoDB. La aparición del indicador `test>` confirma que el servidor aceptó la conexión y que la sesión se encuentra posicionada sobre la base de datos predeterminada `test`.

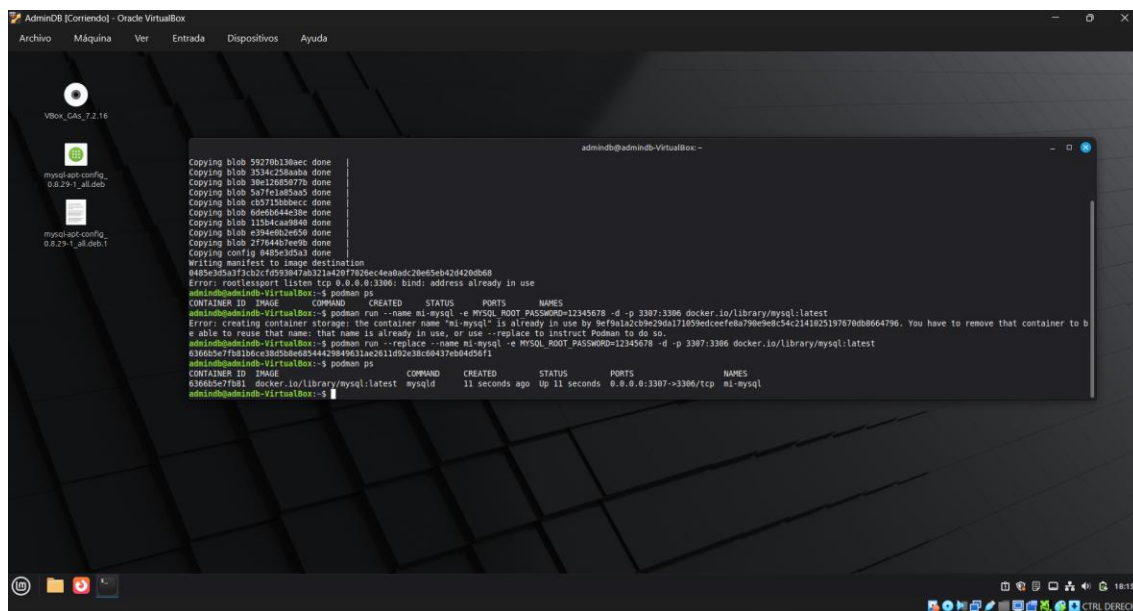


3. Despliegue de Base de Datos Relacional (MySQL) y Resolución de Conflictos

El siguiente paso consistió en levantar una base de datos relacional. Para ello se descargó la imagen oficial de MySQL desde el repositorio `library` de Docker Hub:

```
podman pull docker.io/library/mysql:latest
```

Al intentar ejecutar el contenedor mapeando el puerto 3306 del anfitrión al 3306 del contenedor (`-p 3306:3306`), el sistema arrojó el error `bind: address already in use`. Este mensaje corresponde a la llamada al sistema `bind()`, que es rechazada cuando se intenta asociar un socket a una combinación de dirección IP y puerto que ya está en uso; en TCP, solo un proceso puede permanecer en estado de escucha (`LISTEN`) sobre una misma combinación. En este caso, el puerto 3306 del sistema anfitrión ya se encontraba ocupado, probablemente por una instalación local de MySQL o MariaDB ejecutándose como servicio del sistema operativo. Al no poder establecer el enlace en el anfitrión, Podman no logró iniciar el contenedor.



Para resolver el conflicto se decidió exponer el contenedor en el puerto 3307 de la máquina virtual, enlazándolo al puerto 3306 del contenedor. Esta solución es viable porque cada contenedor opera en su propio espacio de nombres de red: el servicio MySQL en su interior continúa escuchando en su puerto estándar sin interferir con el anfitrión, y únicamente el puerto publicado hacia el exterior debe ser distinto. En consecuencia, los clientes externos deberán conectarse al puerto 3307.

Adicionalmente, el intento fallido ya había creado el contenedor y, con ello, reservado el nombre `mi-mysql`. Dado que los nombres de contenedor deben ser únicos, una nueva ejecución con el mismo nombre habría sido rechazada. Por ello se utilizó la bandera `--replace`, que elimina el

contenedor preexistente con ese nombre antes de crear el nuevo. La solución definitiva se aplicó con el siguiente comando:

```
podman run --replace --name mi-mysql -e MYSQL_ROOT_PASSWORD=12345678 -d -p 3307:3306 docker.io/library/mysql:latest
```

La opción `-e` define la variable de entorno `MYSQL_ROOT_PASSWORD`, requerida por la imagen oficial para inicializar la cuenta `root` del servidor durante el primer arranque. La contraseña empleada es de carácter exclusivamente didáctico y no debe utilizarse en entornos de producción.

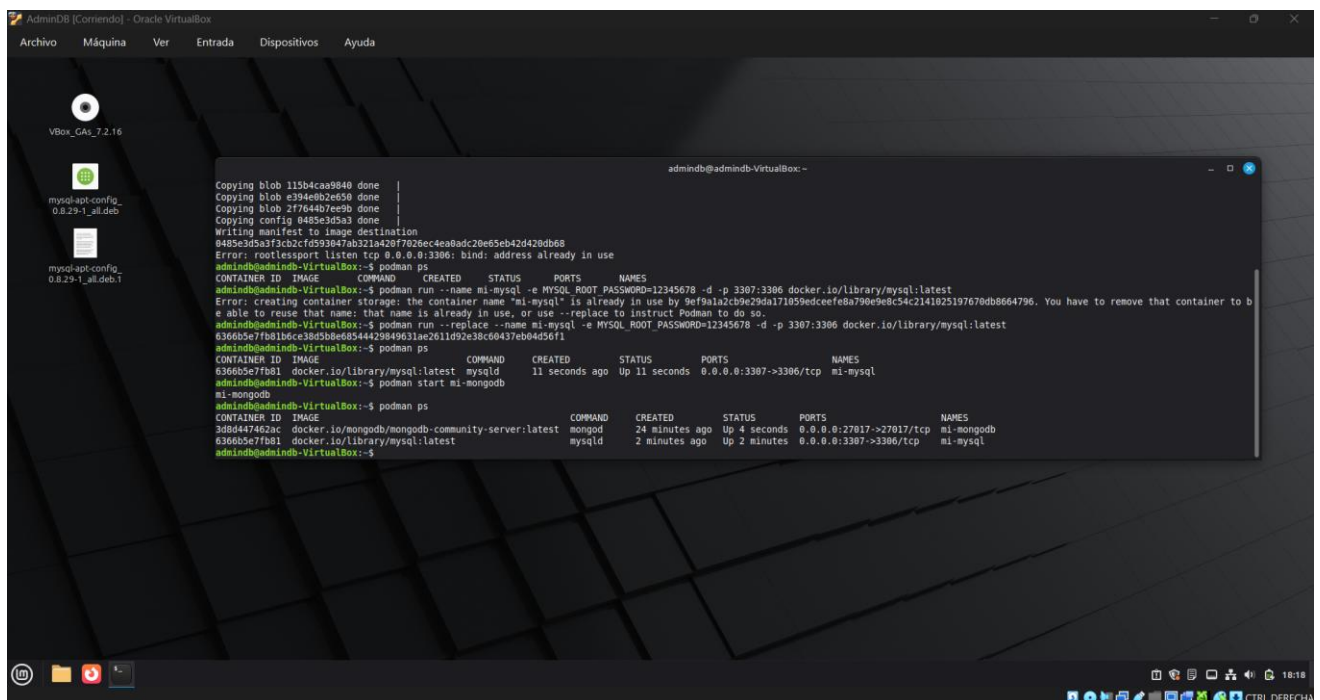
4. Verificación Final y Conclusión

Para concluir la práctica era necesario que ambos contenedores, MongoDB y MySQL, se ejecutaran de forma simultánea. Dado que el contenedor de MongoDB se encontraba detenido, se reinició mediante el siguiente comando, el cual reutiliza el contenedor ya creado sin necesidad de volver a definirlo:

```
podman start mi-mongodb
```

Posteriormente se comprobó el estado de ambos servicios con el comando `podman ps`, que enumera los contenedores en ejecución junto con su identificador, imagen, estado y puertos publicados:

```
podman ps
```



```
admin@admin-VirtualBox: ~$ podman ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
6366b5e7fb81   docker.io/library/mysql:latest      mysqld                  11 seconds ago Up 11 seconds 0.0.0.0:3307->3306/tcp   mi-mysql

admin@admin-VirtualBox: ~$ podman start mi-mongodb
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
3b8d447462ac   docker.io/mongodb/mongodb-community-server:latest  mongod                  24 minutes ago Up 4 seconds 0.0.0.0:27017->27017/tcp  mi-mongodb
6366b5e7fb81   docker.io/library/mysql:latest      mysqld                  2 minutes ago Up 2 minutes 0.0.0.0:3307->3306/tcp   mi-mysql

admin@admin-VirtualBox: ~$
```

La configuración final de los servicios desplegados se resume en la siguiente tabla:

Tabla 1. Mapeo de puertos de los contenedores desplegados

Contenedor	Motor	Tipo de base de datos	Puerto del anfitrión	Puerto del contenedor
mi-mongodb	MongoDB Community	NoSQL orientada a documentos	27017	27017
mi-mysql	MySQL (imagen oficial)	Relacional (SQL)	3307	3306

5. Creación de Base de Datos e Inyección de Script

Para poblar el motor relacional con información de prueba, se inyectó un script estructurado (`tienda.sql`) directamente al contenedor de MySQL. Para lograrlo sin necesidad de abrir una sesión interactiva prolongada, se utilizó el operador de redirección `<` combinado con `podman exec` en modo interactivo (`-i`). Este comando interpretó y ejecutó las instrucciones de creación de tablas e inserción de registros de forma automatizada.

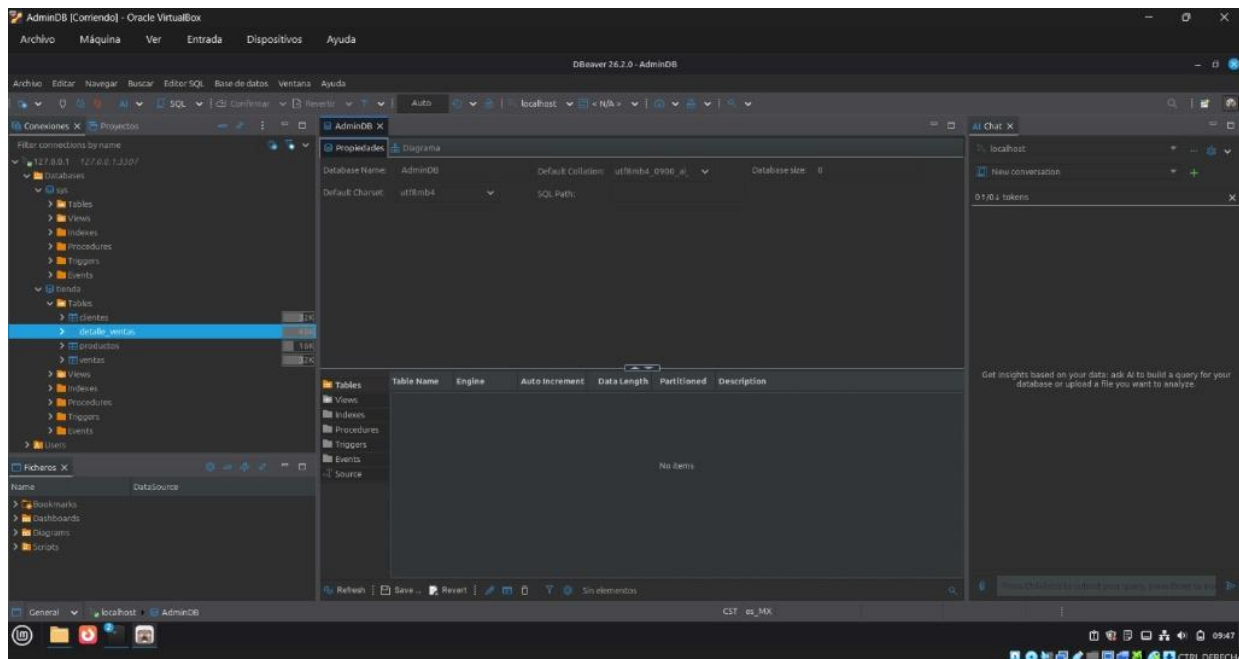
```
admin@adminb-VirtualBox: ~/Descargas
admin@adminb-VirtualBox:~/Descargas$ podman exec -i mi-mysql mysql -u root -p12345678 < tienda.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/run/mysqld/mysqld.sock'
admin@adminb-VirtualBox:~/Descargas$ podman exec -i mi-mysql mysql -h 127.0.0.1 -u root -p12345678 < tienda.sql
mysql: [Warning] Using a password on the command line interface can be insecure.
admin@adminb-VirtualBox:~/Descargas$ podman exec -i mi-mysql mysql -h 127.0.0.1 -u root -p12345678 < tienda.sql
```

La validación de la correcta ejecución del script se llevó a cabo a través de dos métodos. El primero consistió en acceder a la consola interactiva del contenedor y ejecutar la sentencia `SHOW DATABASES;`, comprobando que la base de datos `tienda` figuraba en el listado del servidor.

```
admin@adminb-VirtualBox: ~
mysql> show DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| tienda |
+-----+
5 rows in set (0.006 sec)

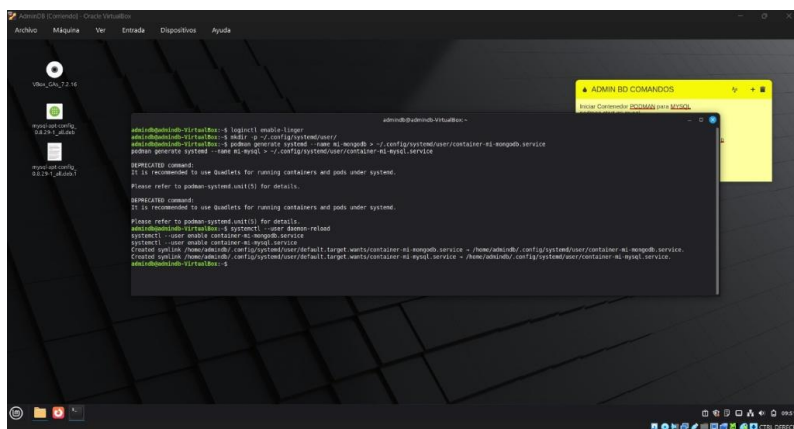
mysql>
```

El segundo método de validación comprobó la accesibilidad externa mediante el cliente gráfico DBeaver. Al conectar la herramienta al puerto expuesto (3307) del anfitrión, fue posible visualizar de manera gráfica la estructura jerárquica de la base de datos tienda y confirmar la creación de sus tablas, tales como `detalle_ventas`.

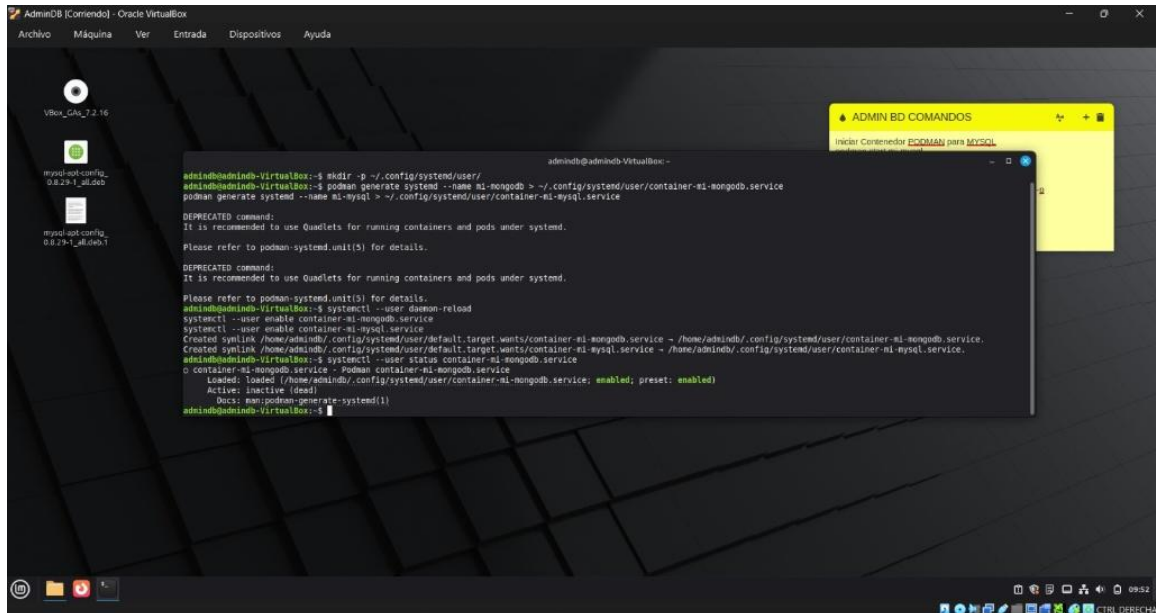


6. Automatización de Arranque con Systemd

Como requerimiento operativo final, se configuró el sistema operativo para que los contenedores se inicien de manera automática al arrancar la máquina virtual. Debido a que Podman opera en modo *rootless* (sin privilegios de superusuario), el primer paso fue habilitar la persistencia de los procesos del usuario utilizando el comando `loginctl enable-linger`. A continuación, se utilizó la herramienta nativa `podman generate systemd` para generar los archivos de servicio unitarios correspondientes a cada contenedor, guardándolos en el directorio de configuración local del usuario.



Finalmente, se recargó el administrador de servicios a nivel de usuario (daemon-reload) y se habilitaron explícitamente los servicios `container-mi-mongodb.service` y `container-mi-mysql.service`. La consola confirmó la creación exitosa de los enlaces simbólicos (symlinks), garantizando que ambos motores de bases de datos se ejecuten en segundo plano de manera transparente tras cada reinicio del equipo.



```
AdminDB [Condomo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

VBox_GAs_7.2.16
mysql.conf.d
0.8.29-1_08.000
mysql.conf.d
0.8.29-1_08.000

admin@adminb-virtualbox:~$ mkdir -p ~/config/systemd/user/
admin@adminb-virtualbox:~$ podman generate system --name mi-mongodb > ~/config/systemd/user/container-mi-mongodb.service
podman generate system --name mi-mysql > ~/config/systemd/user/container-mi-mysql.service

DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.
Please refer to podman-systemd.unit(5) for details.

DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.
Please refer to podman-systemd.unit(5) for details.

admin@adminb-virtualbox:~$ systemctl --user daemon-reload
systemctl --user enable container-mi-mongodb.service
systemctl --user enable container-mi-mysql.service
Created symlink /home/adminb/.config/systemd/user/default.target.wants/container-mi-mongodb.service -> /home/adminb/.config/systemd/user/container-mi-mongodb.service.
admin@adminb-virtualbox:~$ systemctl --user status container-mi-mongodb.service
○ container-mi-mongodb.service - Podman container-mi-mongodb.service
   Loaded: loaded (/home/adminb/.config/systemd/user/container-mi-mongodb.service; enabled; preset: enabled)
   Active: inactive (dead)
   Docs: man:podman-generate-systemd(1)
admin@adminb-virtualbox:~$
```

Continuara...